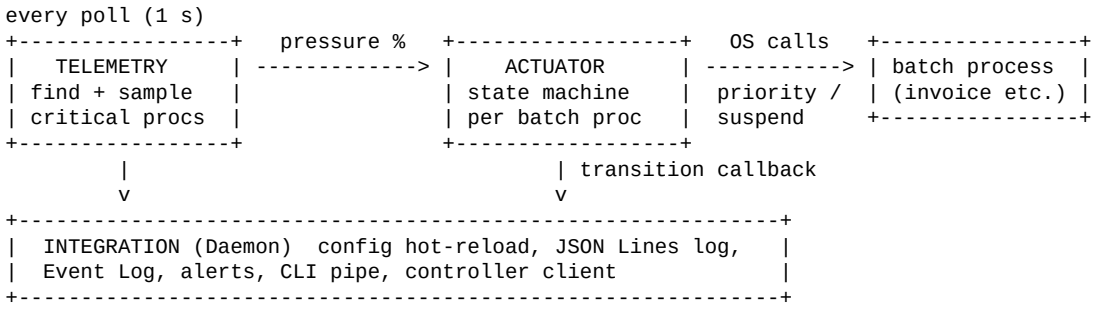
# Actuator / Scheduling Module

Module owner: **Divik** | Logistics server resource monitor & priority throttler (Windows, C++17)

## 1. What this module does

The Actuator is the part of ThreadHaul-Daemon that **acts**. Every second it receives one number, the pressure on the critical process (its CPU%, or memory/I-O pressure when those triggers are on), and for every batch process it decides whether to leave it alone, **demote its priority**, **suspend all its threads** or **restore it**. Each batch process has its own state machine, so one batch job can be suspended while another is only throttled. The module also owns the safety rules (protection list, PID-reuse guard, partial-failure reporting) and the manual override used by the CLI.

## 2. Where it fits in the system




Thresholds (defaults, all configurable): throttle at 85% for 3 samples, suspend at 90% for 5 samples after throttling, recover below 60% for 5 samples, 1 sample per second.

## 3. Files in this module

| File | Purpose |
|---|---|
| Source/Actuator/StateMachine.h/.cpp | ProcessStateMachine: the Normal / Throttled / Suspended state machine for one batch process. |
| Source/Actuator/ActuatorManager.h/.cpp | ActuatorManager: owns one state machine per batch PID, protection list, config and manual-override entry points. |
| Source/Actuator/PriorityControl.h/.cpp | SetProcessPriority(): wraps OpenProcess + SetPriorityClass. IsRunningElevated(). |
| Source/Actuator/ThreadControl.h/.cpp | SuspendAllThreads() / ResumeAllThreads(): thread snapshot + SuspendThread/ResumeThread. |
| Source/Actuator/ThresholdEvent.h | Shared struct ThresholdEvent (contract with Telemetry, INTERFACES.md section 2). |
| Source/main.cpp | Service entry point (ServiceMain, control handler), console mode, CLI commands, SeDebugPrivilege. |
| Source/Tests/StateMachineTests.cpp | 50 unit checks for the state machine and manager. |

## 4. The state machine

| From | Condition | To | OS action |
|---|---|---|---|
| Normal | pressure >= 85% for 3 consecutive samples | Throttled | SetPriorityClass(IDLE_PRIORITY_CLASS) |
| Throttled | pressure >= 90% for 5 consecutive samples (counted after the throttle) | Suspended | SuspendThread on every thread |
| Throttled or Suspended | pressure < 60% for 5 consecutive samples | Normal | ResumeThread on every thread (if suspended) + SetPriorityClass(NORMAL_PRIORITY_CLASS) |
| any | manual command (CLI / controller) | requested state, held | same OS calls; Update() ignores samples while held |

| From | Condition | To | OS action |
|---|---|---|---|
| any | service stop / shutdown | Normal | ForceRecoverForShutdown(): resume + restore |

## 5. Windows APIs and libraries used

| API / library | Where | What it is used for |
|---|---|---|
| OpenProcess(PROCESS_SET_INFORMATION, ...) | PriorityControl.cpp | Gets a handle that is allowed to change the batch process's priority class. |
| SetPriorityClass(h, IDLE_PRIORITY_CLASS / NORMAL_PRIORITY_CLASS) | PriorityControl.cpp | Demotes the batch process so the Windows scheduler runs it only when nothing else wants the CPU, and later restores it. |
| CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0) + Thread32First / Thread32Next | ThreadControl.cpp | Lists every thread in the system; we keep those whose th32OwnerProcessID is the batch PID. |
| OpenThread(THREAD_SUSPEND_RESUME, ...) | ThreadControl.cpp | Gets a handle to each thread with just the right needed to suspend/resume it. |
| SuspendThread / ResumeThread | ThreadControl.cpp | Freeze or unfreeze one thread. Return value -1 = failure, counted in ThreadOpResult. |
| OpenProcess(SYNCHRONIZE, ...) + WaitForSingleObject(h, 0) | StateMachine.cpp | Keeps a handle to each managed process. While we hold it, Windows cannot reuse the PID, and a signalled handle means the process exited (race-condition safety). |
| OpenProcessToken + GetTokenInformation(TokenElevation) | PriorityControl.cpp | IsRunningElevated(): console mode refuses to start unelevated unless --allow-unelevated. |
| AdjustTokenPrivileges(SE_DEBUG_NAME) | main.cpp | Enables SeDebugPrivilege so the service can open processes owned by other accounts. |
| StartServiceCtrlDispatcherW, RegisterServiceCtrlHandlerExW, SetServiceStatus | main.cpp | Turn the exe into a real Windows Service: report START_PENDING / RUNNING / STOP_PENDING / STOPPED and receive STOP and SHUTDOWN. |
| SetConsoleCtrlHandler | main.cpp | Ctrl+C in console mode sets the stop event so shutdown still restores every process. |
| CompareStringOrdinal(..., TRUE) | ActuatorManager.cpp | Case-insensitive process-name comparison for the protection list. |

## 6. Main functions written in this module

| Function | What it does |
|---|---|
| ProcessStateMachine::Update(double cpu) | Counts consecutive high / critical / low samples and calls TransitionTo when a rule is met. Returns true if the state changed. Does nothing while manually held. |
| ProcessStateMachine::TransitionTo(newState, cpu) | Performs the OS calls for the transition, records whether they fully succeeded (actuation_succeeded), resets the counters, then fires the TransitionCallback (used for logging). |
| ProcessStateMachine::ForceState(state, cpu, hold) | Manual override: jumps straight to a state (including Normal->Suspended and Suspended->Throttled) and optionally holds it. |
| ProcessStateMachine::ForceRecoverForShutdown() | Resumes threads and restores NORMAL priority; used on service stop. |
| ProcessStateMachine::HasExited() | True once the batch process ended; ActuatorManager then drops it. |
| ProcessStateMachine::SetConfig(cfg) | Applies new thresholds (hot-reload or adaptive change) without resetting counters. |
| ActuatorManager::AddBatchProcess(name, pid) | Starts managing a batch process unless it is protected (own process, core Windows processes, configured list). |
| ActuatorManager::UpdateAll(pressure) | Feeds one sample to every managed state machine, removing exited ones. Returns true if anything changed. |
| ActuatorManager::OnThresholdEvent(evt) | Entry point from INTERFACES.md section 2; forwards evt.observedCpuPercent to UpdateAll. |
| ActuatorManager::SetProtectedProcessNames(list) | Replaces the configurable protection list; a managed process that becomes protected is restored and released. |

| Function | What it does |
|---|---|
| ActuatorManager::ForceState / ReleaseManualHold / Find / ForEach | Used by the CLI and the status command. |
| SetProcessPriority(pid, level) | Returns Success / AccessDenied / ProcessNotFound / OtherError so callers can report denied access instead of crashing. |
| SuspendAllThreads(pid) / ResumeAllThreads(pid) | Returns ThreadOpResult (attempted / succeeded / failed). A thread that exited between snapshot and OpenThread is not counted as a failure. |
| wmain (main.cpp) | Parses arguments: --console, --config, --allow-unelevated, validate-config, and the CLI verbs throttle / suspend / resume / release / status / stop. |
| DirectAction (main.cpp) | If no daemon is running, the CLI performs throttle / suspend / resume directly, still honouring the protection list. |

## 7. Robustness and safety handled here

• **Protection list:** the daemon's own PID, System, Registry, smss.exe, csrss.exe, wininit.exe, winlogon.exe, services.exe and lsass.exe are always protected, plus *processes.protected* from config. Suspending these could hang Windows.

• **PID reuse / race condition:** each state machine holds a SYNCHRONIZE handle, so the PID cannot be recycled while it is managed; if the process exits mid-suspension the resume step sees HasExited() and skips the OS calls instead of touching an unrelated process.

• **Partial failure:** if SetPriorityClass is denied or not every thread could be suspended, the state still advances (so the machine does not retry forever) but *actuation_succeeded* is logged as false.

• **Escalation counting:** counters reset on every transition, so the 5 critical samples for suspension are counted *after* throttling, exactly as the spec requires.

• **Graceful shutdown:** service STOP, system SHUTDOWN, Ctrl+C and the CLI stop command all end in ForceRecoverAllForShutdown().

## 8. How to demonstrate it

• Build and test: `build.cmd test`; StateMachineTests must print 0 failed.

• Run the console demo (RUNNING.md section 4) and watch batch_simulator switch to IDLE, stop printing, then print 'resumed after a pause'.

• CLI: `ThreadHaulDaemon.exe suspend batch_simulator.exe`, then `status` shows manual_hold true; `suspend explorer.exe` is refused.

## 9. Likely review questions

**Q.** Why IDLE priority first instead of suspending straight away?

**A.** A priority change is cheap and reversible and lets the batch job keep using idle CPU; suspension is a stronger action kept for when demotion is not enough (graded escalation).

**Q.** Why suspend threads rather than kill the process?

**A.** Batch work (invoices, sync) must not be lost; suspended threads continue exactly where they stopped.

**Q.** What if a thread is created after the snapshot?

**A.** It would not be suspended in that pass; ResumeAllThreads takes a fresh snapshot so nothing is left suspended. This is a known limitation of SuspendThread-based freezing.

**Q.** How does the CLI avoid fighting the automatic logic?

**A.** ForceState(..., hold=true) sets manualHold_; Update() keeps counting but never transitions until 'release'.

See also: README.md (overview), RUNNING.md (how to run), INTERFACES.md (cross-module contract), Source/Docs/literature-review.md (design rationale).